

라이브러리 작성 기법

- 라이브러리란? 1.
- 정적라이브러리 2.
- 공유라이브러리 3.
- 동적라이브러리 4.

1. 라이브러리란?

1.1 라이브러리란?

라이브러리란? 함수의 집합을 라이브러리라 한다. 라이브러리의 분류는 시스템에서 미리 제공되는 라이브러리와 사용자가 정의한 라이브러리로 분류할 수 있다. 시스템에서 제공하는 라이브러리의 대표적인 예는 수학 함수 라이브러리와 그래픽 라이브러리등이다.

라이브러리를 작성하고 사용하는 방법에 따라 크게 3 가지로 구분할 수 있다.

- 정적 라이브러리
- 공유 라이브러리
- 동적 라이브러리

2. 정적 라이브러리

2.1 정적 라이브러리

정적 라이브러리는 단지 오브젝트 화일을 모아놓은 것으로, 일반적으로 정적 라이브러리 파일명은 .a라는 확장자를 이용한다.

이 오브젝트 집합은 ar(archiver) 프로그램에 의해서 생성된다. 정적 라이브러리는 공유 라이브러리가 가지는 장점들 때문에 이제는 옛날처럼 그렇게 많이 사용되지는 않지만 종종 사용되기도 한다.

정적 라이브러리는 재컴파일 없이 프로그램에 링크될 수 있기 때문에 재컴파일에 필요한 시간이 절약될 수 있다. 그러나 컴파일시 실행 파일에 적재되므로 유연성이 떨어지며, 실행파일의 크기가 커지는 문제가 있다. 요즘은 컴퓨터의 빠른 실행시간 때문에 이런 재컴파일에 드는 시간 절약은 옛날처럼 그렇게 중요한 이유가 될 수 는 없다.

정적 라이브러리는 개발자들이 소스코드의 제공 없이 프로그래머들로 하여금 링크하여 사용하도록 할 때는 유용하다

정적 라이브러리 작성 절차

1. 함수가 선언될 header 파일을 생성 한다.
2. 함수가 구현될 파일을 생성한다.
3. 함수가 구현된 파일을 컴파일 한다.
4. ar(archive) 명령어를 이용하여 정적 라이브러리 파일을 생성한다.
5. 작성된 라이브러리 파일을 사용하는 예제 파일을 작성한다.
6. 예제 파일에 작성된 라이브러리를 적용한다.
7. 예제 파일을 실행한다.

[예제 4-1] Header 파일 생성 (calc.h)

단계1. 함수가 선언될 header 파일을 생성한다.

```

/*****
filename : calc.h
*****/
/*해더 파일이 중복 되어 include 되지 않게 하기 위한 조건 컴파일*/
#ifndef _CALC_H_
#define _CALC_H_

/*현재 작성되는 정적 라이브러리 함수가 c++에서 사용될 때 함수 이름이 c언어에서 생성되는 구조로
만들어준다.*/
#ifdef __cplusplus
extern "C" {
#endif

/*함수 원형을 선언한다.*/
void add(int a, int b);
void sum(int a, int b);
void mul(int a, int b);
void div(int a, int b);

#ifdef __cplusplus
}
#endif

#endif //_CALC_H_

```

단계2. 함수가 구현될 파일을 생성한다. (add.c, sub.c, mul.c, div.c)

단계3. 함수가 구현된 파일을 컴파일한다.

각 구현된 파일을 object로 생성한다. object 파일로 생성할 때는 컴파일 옵션에 -c 만을 주면 된다.

```
$ gcc -c add.c sub.c mul.c div.c
```

생성된 object 파일 목록

```
$ls -l *.o
-rw-rw-rw- 1 unix00    unix      864 Oct 14 12:05 add.o
-rw-rw-rw- 1 unix00    unix      980 Oct 14 12:06 div.o
-rw-rw-rw- 1 unix00    unix      912 Oct 14 12:05 mul.o
-rw-rw-rw- 1 unix00    unix      864 Oct 14 12:05 sub.o
```

단계4. ar(archive) 명령어를 이용하여 정적 라이브러리 파일을 생성한다.

정적라이브러리는 여러 개의 object 파일을 묶어놓고 필요할 때마다 쉽게 꺼내어 사용하기 위해 만들어 놓는 것을 말하는데 ar라는 유틸리티를 사용한다.

특히 ar은 서브루틴(함수)을 호출하기 위해 만들어 놓은 목적 파일을 하나의 라이브러리로 묶는 데 자주 사용되므로 바이너리 유틸리티라고 부르기도 한다. archive 파일에 어떠한 유형의 파일이라도 추가, 삭제하고 변경할 수 있습니다.

형식

ar [-][옵션][멤버파일] 아카이브 파일들

아래는 옵션입니다.

```
-d : archive 로 부터 파일을 삭제한다.
-q : 이미 존재하고 있더라도 archive 의 끝에 파일을 추가합니다.
-r : 파일이 archive 내에 존재하지 않으면 추가하고, 존재하면 새 버전으로 변경합니다.
-t : 표준 출력으로 archive의 목차를 출력합니다.
-x : archive 로 부터 파일들을 현재의 디렉토리로 복사합니다.
-v : 출력을 확인하고 싶을 경우 사용한다,
```

생성된 object 파일을 archive 파일에 추가합니다.

```
$ ar -rv libcalc.a add.o sub.o mul.o div.o
```

archive 파일에 기록된 파일 목록을 확인한다.

```
$ ar -tv libcalc.a
```

```
rw-rw-rw-  493/202      864 Oct 14 12:05 2006 add.o
rw-rw-rw-  493/202      864 Oct 14 12:05 2006 sub.o
rw-r----- 493/202      268 Oct 14 12:04 2006 mul.c
rw-r----- 493/202      357 Oct 14 12:06 2006 div.c
```

단계5. 작성된 라이브러리 파일을 사용하는 예제 파일을 작성한다.

```
/*
filename : calc.c
*/
#include <stdio.h>
#include "calc.h"

int main(int argc, char* argv[])
{
    int a, b;

    printf("a = ");
    scanf("%d", &a);
    printf("b = ");
    scanf("%d", &b);

    add(a, b);
    sub(a, b);
    mul(a, b);
    div(a, b);

    return 0;
}
```

단계6. 예제 파일에 작성된 라이브러리를 적용한다.

작성한 파일을 컴파일 한다.

```
$ gcc -c calc.c
```

라이브러리를 링크시 명시해 주는 옵션은 `-l` 이다. `-l`(소문자) 뒤에 공백 없이 라이브러리명을 지정한다. 라이브러리명을 사용할 때는 앞에 있는 접두어와 확장자를 제거하고 사용한다. 예를 들어 라이브러리명이 `libcalc.a` 이면 `-lcalc` 라고 명시한다.

```
$ gcc -o calc calc.o -lcalc
```

위와 같이 실행하면 `-lcalc` 라이브러리 파일을 찾을 수 없다는 오류가 발생한다. 기본적으로 링크시 라이브러리는 `/lib`, `/usr/lib`, `/usr/local/lib` 경로 있는 파일만을 찾아서 링크를 하기 때문이다. 따라서 사용자가 작성한 라이브러리를 링크할 때에는 참조할 수 있게 경로를 명시해야 한다. 이때 사용하는 옵션이 `-L`(대문자) 이다.

```
$ gcc -o calc calc.o -L . -lcalc
```

`-L .` 은 현재의 경로를 의미한다.

단계8. 예제 파일을 실행한다.

```
$ ./calc
a = 10
b = 3
static library calc : add() 호출
10 + 3 = 13
static library calc : sub() 호출
10 - 3 = 7
static library calc : mul() 호출
10 * 3 = 30
static library calc : div() 호출
10 / 3 = 3
```


이때 사용되는 각 함수는 archive에 의해 작성된 정적 라이브러리를 사용한다.

만약 `add()` 함수를 변경한다면 `add.c` 파일을 컴파일하고 `ar` 명령을 사용하여 이전 파일을 새로운 파일로 변경한 후 `calc` 파일을 다시 생성해야 한다.

3. 공유 라이브러리

3.1 공유 라이브러리

공유라이브러리는 실행시 로딩 되는 라이브러리를 말하며, 이 공유 라이브러리는 최초 실행시 메모리에 로딩된 후 다른 응용프로그램에 의해 사용될 때 이미 메모리에 로딩된 라이브러리가 사용된다. 즉 기존 응용프로그램에 추가되는 것이 아니라 실행시 링크가 된다. 그래서 기존에 있는 공유라이브러리를 새로운 기능으로 수정하면 기존에 있던 응용프로그램도 자동으로 버전이 업그레이드 된다. 이런 공유라이브러리를 사용하는 응용프로그램을 배포할 때는 응용프로그램과 공유라이브러리도 같이 배포를 해야한다. 응용프로그램이 어떤 공유 라이브러리를 사용하는지 알아보는 명령어이 ldd 명령이다.

ldd 사용예

\$ldd /bin/sh

```
/usr/lib/libc.2 => /usr/lib/libc.2
/usr/lib/libdld.2 => /usr/lib/libdld.2
/usr/lib/libc.2 => /usr/lib/libc.2
/usr/lib/libxti.2 => /usr/lib/libxti.2
/usr/lib/libnsl.1 => /usr/lib/libnsl.1
/usr/lib/libxti.2 => /usr/lib/libxti.2
```

windows 환경에서 공유 라이브러리를 생성하기 위해서는 header 파일과 소스를 수정해야한다. 그러나 unix/linux 환경에서는 소스를 변경할 필요가 없다. 컴파일과 링크만 다시 하면 공유 라이브러리를 생성할 수 있다.

공유 라이브러리 작성 예

단계 1. 기존에 생성된 object 파일과 실행 파일을 삭제합니다.

```
$ rm *.o libcalc.a calc
```

단계 2. 공유 라이브러리를 생성하기 위해서는 object 파일 생성시 -fPIC 을 반드시 명시 해야 한다. -fPIC 옵션은 독립적인 코드를 생성해 주는 옵션이다.

object 파일 생성

```
$ gcc -fPIC -c add.c
$ gcc -fPIC -c sub.c
$ gcc -fPIC -c mul.c
$ gcc -fPIC -c div.c
```

공유라이브러리를 생성할 때는 파일명 앞에 접두어로 lib를 붙이고 확장자는 .so 를 사용한다. 그리고 링크 옵션으로 -shared를 명시해준다. -shared 옵션은 작성된 라이브러리가 공유 라이브러리라는 것은 의미한다.

단계 1. 공유 라이브러리 생성

```
$ ld -shared -o libcalc.so add.o sub.o mul.o div.o
```

단계 2. 공유 라이브러리 참조

```
$ gcc -o calc calc.o -L. -lcalc
```

단계 3. 프로그램 실행

만약 공유 라이브러리를 찾을 수 없다는 오류가 발생하면 공유 라이브러리를 찾는 환경설정 변수에 공유 라이브러리가 존재하는 경로를 추가해야 한다.

```
$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:. ←라이브러리 파일 위치
```

위 명령을 환경 설정 파일 .bashrc에 추가하면 된다.

단계 4. 이제 add.c 파일을 수정하여 add() 함수의 기능을 업그레이드 해본다.

```

/*****
filename : add.c
*****/
#include <stdio.h>

/*add () 함수를 구현합니다*/
void add(int a, int b)
{
    printf("shared library calc : add() 호출 \n");

    printf("%d + %d = %d\n", a, b, a + b);
    printf("%d * %d = %d\n", a, b, a * b);
}

```

단계 5. 컴파일을 다시 한다.

```
$ gcc -fPIC -c add.c
```

단계 6. 공유 라이브러리를 다시 작성한다

```
$ ld -shared -o libcalc.so add.o sub.o mul.o div.o
```

단계 7. 프로그램을 다시 실행한다.

4. 동적 라이브러리

4.1 동적 라이브러리

동적 라이브러리는 프로그램이 시작할 때가 아닌 실행 중에 로딩되는 라이브러리이다. 이 기능은 플러그인이나 모듈을 구현할 때 적합하다. 왜냐하면 그것들이 필요할 때까지 로딩을 기다릴 수 있기 때문이다. 이 기능은 때때로 코드를 머신 코드로 바꾸고 효율을 위해 멈추지 않고 컴파일된 코드로 만드는 인터프리터를 구현하는데 유용하다. 이것은 실시간의(just-in-time) 컴파일러나 MUD(multi-user dungeon)을 구현하는데 유용하다.

unix/linux 에서, 동적 라이브러리는 포맷 관점에 있어서 특별하지 않다. 그들은 표준의 오브젝트 파일로 만들어지거나, 위에서 언급된 표준의 공유 라이브러리 파일로 만들어진다. 주요 차이점은 프로그램의 링크시나 시작시에 로딩되지 않는다는 것이다; 대신, 라이브러리를 열거나, 심볼을 찾거나, 에러를 조정하거나, 라이브러리를 닫는 특별한 API가 있다. C 사용자들은 이런 API를 사용하기 위해 dlfcn.h라는 헤더파일을 포함해야 한다.

unix/linux에서는 일련의 'dlopen()' API를 이용하여 동적 라이브러리 기능을 구현한다. 그러나, 이 API가 모든 플랫폼에서 지원되는 것은 아니다. 예를 들어 HP-UX는 shl_load()를 사용하고, 윈도우즈는 완전히 다른 인터페이스의 DLLs를 사용한다. 만약 폭넓은 포팅성이 목표라면, 다른 플랫폼을 숨기는 라이브러리를 생각해야한다.

4.2 dlopen() 함수

dlopen(3)함수는 동적 라이브러리를 로딩하고 사용가능하도록 준비시켜준다. C에서 프로토타입은 다음과 같다.

```
#include <dlfcn.h>
void * dlopen(const char *filename, int flag);
```

파일 이름이 '/' (즉, 절대경로)로 시작한다면 dlopen()은 즉시 로딩가능하다(라이브러리를 찾

으려 하지 않는다). 그렇지 않으면, `dlopen()`은 다음과 같은 순서로 라이브러리를 찾는다.

- 사용자의 `LD_LIBRARY_PATH`의 환경변수의 콜론으로 구분지어진 디렉토리
- (`/etc/ld.so.conf`에서 파생된) `/etc/ld.so.cache`에 명시되어있는 라이브러리의 목록

`/lib`, `/usr/lib` 순서를 주의해야한다. 보통은 어떤 디렉토리나 다른 디렉토리 하나에만 라이브러리가 있기 때문에(둘다 없을 수도 있다) 문제가 되지 않는다. 만약 여러 디렉토리에 동일한 이름을 가진 라이브러리(다른 라이브러리)가 있다면 잠재적인 위험을 가지게 된다.

`dlopen()`에서 `flag`의 값은 다음과 같다.

`RTLD_LAZY` : 실제로 사용되는 시점에 로딩(`dlsym()`에서 함수 등을 바인드한 시점이 되었을 때)함. 정의되지 않은 심볼이 있으면 알아볼 수 없는 에러 발생한다.

`RTLD_NOW` : `dlopen()`에서 로딩 완료

`RTLD_GLOBAL` : 라이브러리에 적재되는 외부 심볼들은 그 후에 적재되는 라이브러리에 의해서 사용 가능

동적 라이브러리들이 서로 의존한다면(예를 들어 X가 Y에 의존한다면), 의존 당하는 것을 먼저 로드해야 한다

`dlopen()`의 리턴 값은 성공시 ‘핸들’을, 실패하면 `NULL`을 리턴한다. `dlopen()`에 의해 같은 라이브러리가 한번 이상 로드되면 같은 핸들이 리턴된다.

4.3 `dlerror()` 함수

에러는 `dlerror()`를 호출함으로써 확인할 수 있다. `dlerror()`는 `dlerror()`, `dlsym()`, `dlclose()`중 마지막 호출한 에러의 문자열을 반환한다.

4.4 `dlsym()` 함수

공유 라이브러리를 사용하는 중요한 함수는 `dlsym(3)`이다. 이것은 공유 라이브러리의 함수명에 대한 함수의 주소를 리턴해준다. 이 함수는 다음과 같이 정의된다:

```
void *dlsym(void *handle, char *symbol);
```

핸들은 `dlopen`에 의해 반환된 값이고, 심볼은 찾고자 하는 함수명이다. `dlsym()`은 심볼이 없으면 `NULL`을 반환한다. 만약 심볼에 `NULL`이나 `0`이 없다는 것을 알면 좋겠지만, 다음과 같은 잠재적인 문제가 있다: 반환값이 `NULL`인가, 아니면 심볼의 값이 `NULL`인가?

표준적인 해결법은 `dlerror()`를 부르고(전에 있던 에러들을 없앤다), `dlsym()`을 불러서 심볼의 값을 부르고, `dlerror()`를 불러서 에러가 났는지 체크한다. 단순한 코드는 다음과 같다.

```
dlerror(); /* clear error code */
s = (actual_type)dlsym(handle, "원하는 함수명");
if ((err = dlerror()) != NULL) {
    /* handle error, the symbol wasn't found */
} else {
    /* symbol found, its value is in s */
}
```

4.5 `dlclose()` 함수

`dlclose()` 함수는 공유 라이브러리를 닫아주는 함수로, 공유 라이브러리가 동적 파일 핸들의 링크 수를 관리하기 때문에, 동적 라이브러리는 `dlopen`이 성공한 만큼 모두 `dlclose`를 불러주지 전에는 없어지지 않는다. `dlclose()`는 성공하면 `0`을 리턴하고, 아니면 `0`이 아닌 값을 리턴한다

[동적 라이브러리 예제] (`calc2.c`)

■ 컴파일 하는 방법

동적 라이브러리는 시스템에 따라 `-ldl` 라이브러리 옵션을 사용해야 한다.

실행시 사용하는 동적 라이브러리는 링크시 라이브러리 옵션을 추가 하지 않는다.

```
$ gcc -o calc2 calc2.c -ldl
```

공유 라이브러리 사용 확인

```
$ ldd calc
```

```
    /usr/lib/libc.2 =>      /usr/lib/libc.2
    /usr/lib/libdld.2 =>    /usr/lib/libdld.2
    /usr/lib/libc.2 =>      /usr/lib/libc.2
    ./libcalc.so => ./libcalc.so (공유 라이브러리)
    /usr/lib/libc.2 =>      /usr/lib/libc.2
```

```
$ ldd calc2
```

```
    /usr/lib/libc.2 =>      /usr/lib/libc.2
    /usr/lib/libdld.2 =>    /usr/lib/libdld.2
    /usr/lib/libc.2 =>      /usr/lib/libc.2
```

libcalc.so 공유 라이브러리는 실행시 로딩되므로 표시되는 않는다